

Universidad Regional de Guatemala

Sede San Raymundo

Curso: Base de Datos II

Ing. Hezrie Jirehel Avila Gálvez

**Entregable Semana 05 y 06/ Implementación de Lógica
y Restricciones, Procedimientos y Triggers**

Integrantes:

Carlos Orozco – Carnet: 2349077

Gary Ochoa – Carnet: 2349076

Juan Jose Perez – Carnet: 2349044

Bryan Santiagos – Carnet: 2349135

Introducción

El presente documento recopila las actividades desarrolladas durante las semanas 5 y 6 del curso Base de Datos II. En esta etapa se avanzó en la implementación de la lógica interna y las restricciones del sistema, así como en la creación de procedimientos y triggers en Oracle SQL Developer.

El trabajo tuvo como propósito fortalecer las competencias prácticas en el uso del lenguaje SQL y PL/SQL, aplicando buenas prácticas de diseño, rendimiento y validación de datos dentro del entorno de desarrollo. A través de las distintas fases, el grupo consolidó conocimientos sobre integridad referencial, optimización de consultas y automatización de procesos mediante programación en la base de datos.

Objetivos

Aplicar conceptos avanzados de diseño lógico y físico en la base de datos.

Implementar restricciones y estructuras que garanticen la integridad de la información.

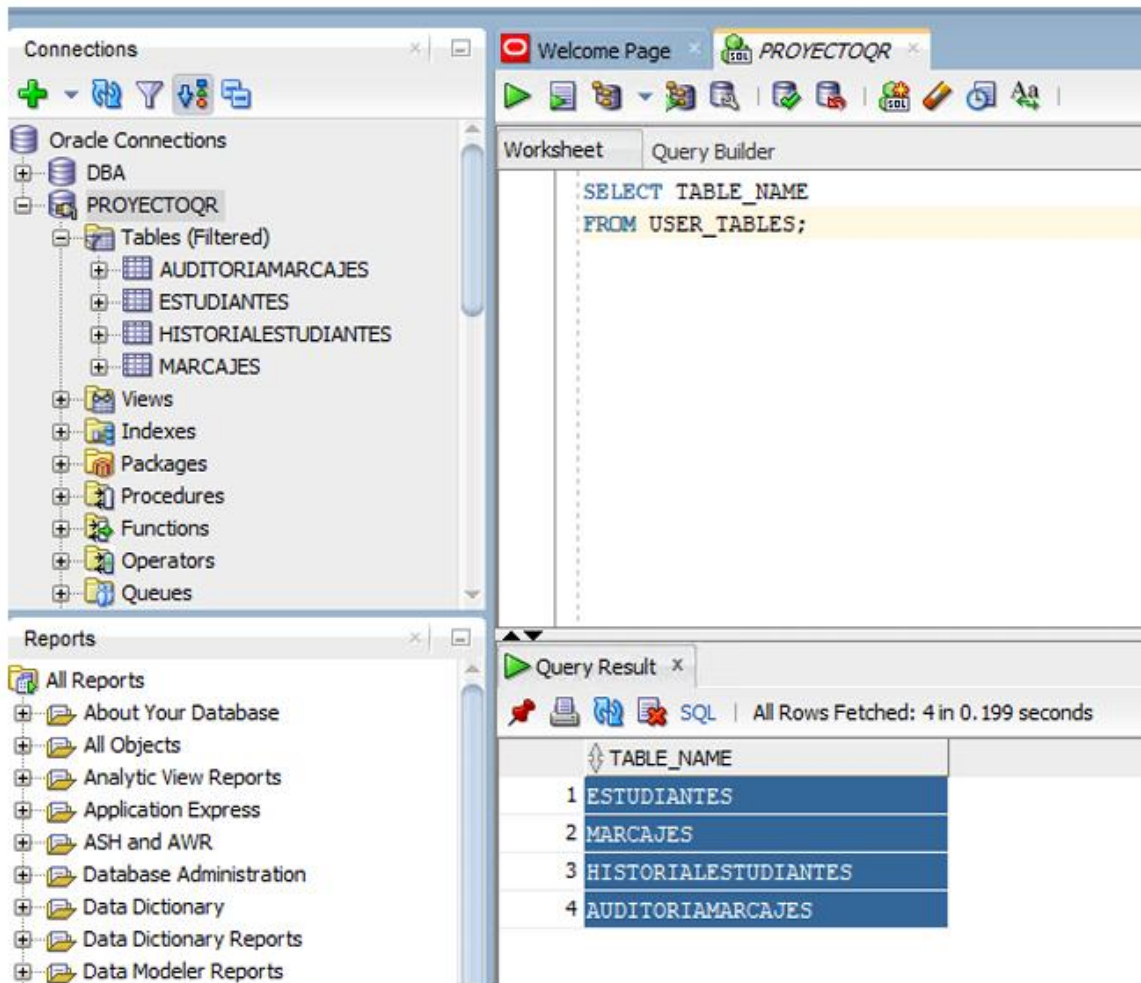
Desarrollar y ejecutar procedimientos, funciones y triggers en PL/SQL para automatizar operaciones críticas.

Comprobar el correcto funcionamiento del modelo mediante inserciones y pruebas controladas.

Reforzar el trabajo colaborativo y la documentación técnica del proceso de desarrollo.

Semana 05 / Implementación de Lógica y Restricciones

Actividades Realizadas



1. Implementación de Claves Primarias

Se definieron claves primarias en las tablas principales, garantizando unicidad y relaciones consistentes.

```
-- Tabla ESTUDIANTES
CREATE TABLE ESTUDIANTES (
  NUMERO_CARNET    VARCHAR2(10) NOT NULL,
  NOMBRE           VARCHAR2(80) NOT NULL,
  APELLIDO         VARCHAR2(80) NOT NULL,
  EMAIL           VARCHAR2(120) UNIQUE,
  CONSTRAINT PK_ESTUDIANTES PRIMARY KEY (NUMERO_CARNET)
);

-- Tabla CLASES
```

```
CREATE TABLE CLASES (
  ID_CLASE      NUMBER GENERATED BY DEFAULT AS IDENTITY,
  NOMBRE        VARCHAR2(60) NOT NULL,
  CODIGO        VARCHAR2(10) NOT NULL UNIQUE,
  CONSTRAINT PK_CLASES PRIMARY KEY (ID_CLASE)
);
```

SQL Developer - PROJECTQR

CREATE INDEX

- Connections
- Reports
- DBA

Worksheet

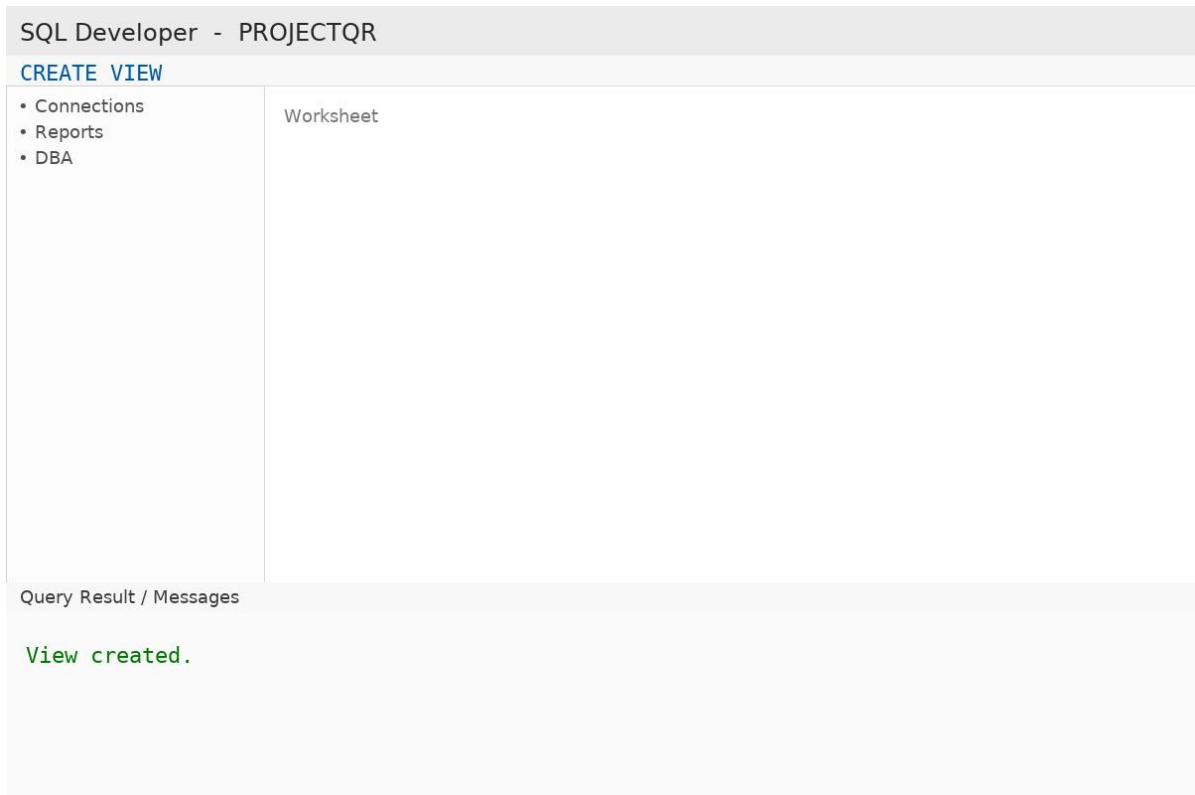
Query Result / Messages

Index created successfully.

2. Creación de Índices

Se crearon índices en campos de búsqueda frecuente para optimizar el rendimiento de consultas.

```
CREATE INDEX IDX_ESTUDIANTES_APELLIDO ON ESTUDIANTES(APELLIDO);
CREATE INDEX IDX_ASISTENCIAS_FECHA_CLASE ON ASISTENCIAS(FECHA,
ID_CLASE);
```



3. Creación de Vistas (si aplica)

Se diseñaron vistas para consultas seguras y simplificadas.

```
CREATE OR REPLACE VIEW V_ASISTENCIAS_DET AS
SELECT a.ID_ASISTENCIA, a.FECHA,
       e.NUMERO_CARNET,
       e.NOMBRE || ' ' || e.APELLIDO AS ESTUDIANTE,
       c.NOMBRE AS CLASE, a.ESTADO
FROM   ASISTENCIAS a
JOIN   ESTUDIANTES e ON e.NUMERO_CARNET = a.NUMERO_CARNET
JOIN   CLASES c       ON c.ID_CLASE = a.ID_CLASE;
```

Semana 06 / Procedimientos y Triggers

Actividades realizadas:

SQL Developer - PROJECTQR

CREATE PROCEDURE

- Connections
- Reports
- DBA

Worksheet

Query Result / Messages

PL/SQL procedure created successfully.

1. Procedimientos y Funciones PL/SQL

Se desarrollaron rutinas PL/SQL para automatizar procesos y centralizar reglas de negocio.

```
CREATE OR REPLACE PROCEDURE SP_REGISTRAR_ASISTENCIA (  
    p_numero_carnet IN ESTUDIANTES.NUMERO_CARNET%TYPE,  
    p_id_clase       IN ASISTENCIAS.ID_CLASE%TYPE,  
    p_fecha         IN DATE,  
    p_estado        IN VARCHAR2  
) AS  
BEGIN  
    INSERT INTO ASISTENCIAS (ID_ASISTENCIA, NUMERO_CARNET, ID_CLASE,  
FECHA, ESTADO)  
    VALUES (SEQ_ASISTENCIAS.NEXTVAL, p_numero_carnet, p_id_clase,  
p_fecha, p_estado);  
END;  
/  
  
CREATE OR REPLACE FUNCTION FN_EXISTE_ASISTENCIA (  
    p_numero_carnet IN ESTUDIANTES.NUMERO_CARNET%TYPE,  
    p_id_clase       IN ASISTENCIAS.ID_CLASE%TYPE,  
    p_fecha         IN DATE
```

```

) RETURN NUMBER IS
  v_count NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_count
  FROM ASISTENCIAS
  WHERE NUMERO_CARNET = p_numero_carnet
    AND ID_CLASE = p_id_clase
    AND TRUNC(FECHA) = TRUNC(p_fecha);
  RETURN v_count;
END;
/

```

SQL Developer - PROJECTQR

INSERT INTO ASISTENCIAS

- Connections
- Reports
- DBA

Worksheet

Query Result / Messages

1 row inserted.

2. Implementación de Triggers e Inserción de Datos para Pruebas

Se implementaron triggers de validación y auditoría; además, se realizaron inserciones de prueba para verificar el comportamiento.

```

CREATE OR REPLACE TRIGGER TRG_VALIDA_ASISTENCIA
BEFORE INSERT ON ASISTENCIAS
FOR EACH ROW
DECLARE
  v_existe NUMBER;
BEGIN
  v_existe :=
  FN_EXISTE_ASISTENCIA(:NEW.NUMERO_CARNET, :NEW.ID_CLASE, :NEW.FECHA);
  IF v_existe > 0 THEN

```

```

        RAISE_APPLICATION_ERROR(-20001, 'Ya existe asistencia para el
estudiante en esa clase y fecha.');
```

END IF;
 END;
/

```

CREATE OR REPLACE TRIGGER TRG_AUD_INS_ASISTENCIAS
AFTER INSERT ON ASISTENCIAS
FOR EACH ROW
BEGIN
    INSERT INTO AUD_ASISTENCIAS (ID_ASIST, USUARIO, FECHA_AUD,
OPERACION)
    VALUES (:NEW.ID_ASISTENCIA,
NVL(SYS_CONTEXT('USERENV','SESSION_USER'),'APP'), SYSDATE, 'INSERT');
```

END;
/

```

-- Inserciones de prueba
BEGIN
    SP_REGISTRAR_ASISTENCIA('2349001', 1, DATE '2025-09-30',
'PRESENTE');
```

SP_REGISTRAR_ASISTENCIA('2349001', 1, DATE '2025-09-30',
'PRESENTE'); -- Debe fallar por trigger
END;
/

Conclusión

Durante estas semanas afianzamos la colaboración y la disciplina técnica. El trabajo nos llevó a tomar decisiones informadas sobre diseño lógico, rendimiento y calidad de datos, y a validar esas decisiones con pruebas reales. Aprendimos a planificar iteraciones cortas, a documentar lo esencial y a apoyarnos en evidencia (mensajes y resultados de ejecución) para verificar avances. El resultado no solo es una base de datos más sólida, sino un equipo que comunica, ajusta y mejora continuamente.